**Artificial Neural Networks**

Neural networks are interconnected assemblies of simple processing elements, called neurons, units or nodes, whose functionality is loosely based on the biological neuron. These networks 'learn' from examples (as children learn to recognize objects) and exhibit some capability for generalization beyond the training data.
The processing ability of the network is stored in the inter-unit connection strength, or weights, obtained by a process of adaptation to, or learning from a set of training patterns. By emulating the way interconnected brain cells function, Neural Network (NN) enabled machines are now trained to learn, recognize patterns, and make predictions in a humanoid fashion as well as solve problems in various fields.

How do humans recognize objects?
As a child when we first see a car, our brain creates a very simple mental model of what a car is.

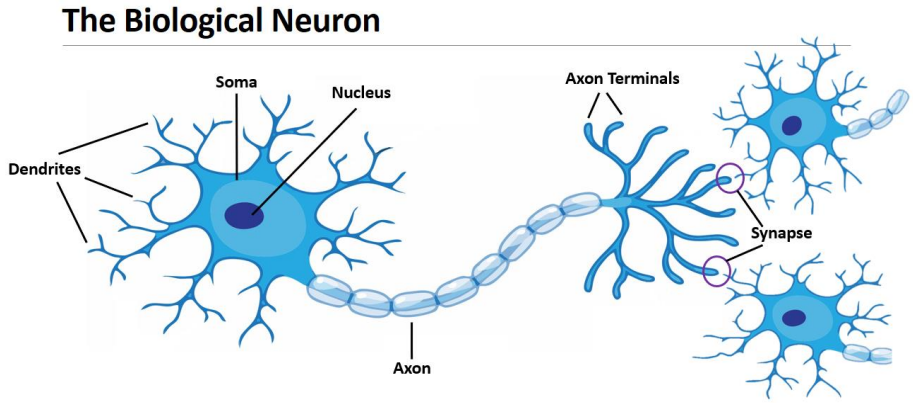
It has four wheels.
It has windows.
It has doors.
It moves on the road.

This is only a rough understanding. It is not complete, but it is enough to recognize something similar. Over time and by looking at several examples of different variations of a car, our brain no longer depends on one specific example. Instead, it learned the important patterns that define a car.
Every new experience slightly improves our internal model. Whenever we make a mistake, someone corrects us. Whenever we see a new type of car, our understanding becomes richer.

**Learning is the gradual refinement of an internal model through experience**

The inspiration for NN came from a desire to produce artificial systems capable of sophisticated and 'intelligent' computations similar to those that the human brain routinely performs

**The Biological Neuron**



Soma or Cell Body

The central part of a neuron is called the soma or cell body which contains the nucleus and the protein synthesis machinery. The size of soma of a typical neuron is about 10 to 80µm. Almost all the logical functions are realized in this part of a neuron.

The Dendrites

Dendrites represent a highly branching tree of fibers and are attached to the soma. The word dendrite has been taken from the Greek word dendro which means tree. Dendrites connect the neuron to a set of other neurons. Dendrites either receive inputs from other neurons or connect other dendrites to the synaptic outputs.

The Axon

It is a long tubular fiber which divides itself into a number of branches towards its end. Its length can be from 100 µm to 1 m. The function of an axon is to transmit the generated neural activity to other neurons or to muscle fibers. In other words, it is output channel of the neuron. The point where the axon is connected to its cell body is called the Hillock zone.

Synapse

The junction at which a signal is passed from one neuron to the next is called a synapse (from the Greek verb to join). It has a button like shape with diameter around 1 m. Usually a synapse is not a physical connection (the axon and the dendrite do not touch) but there is a gap called the synaptic cleft that is normally between 200Å to 500Å (1Å = $10^{-10}$ m). The strength of synaptic connection between neurons can be chemically altered by the brain in response to favorable and unfavorable stimuli in such a way as to adapt the organism to function optimally within its environment.

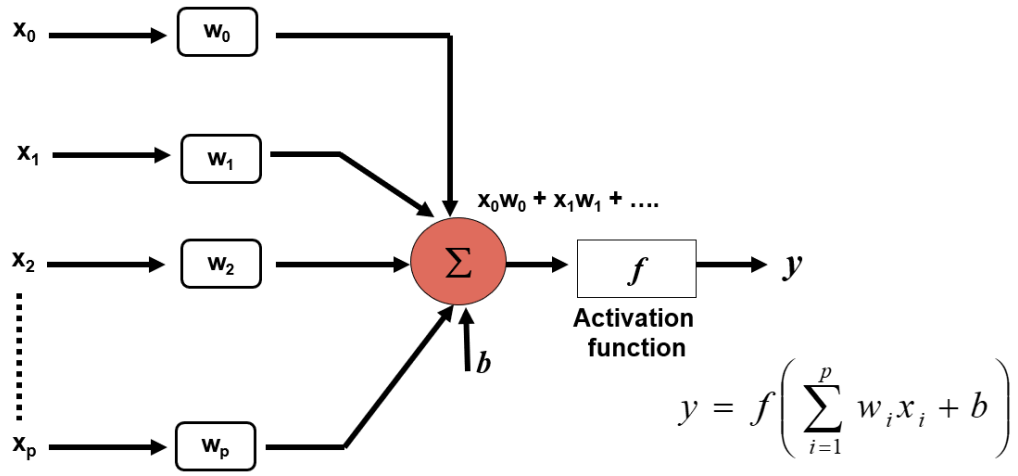
**Is it possible to simulate the human brain?**

No, with present computing power it is difficult to simulate a full behavior of human brain.

Reasons:

It is estimated that the human cerebral cortex contains 100 billion neurons. Each neuron has as many as 1000 dendrites and, hence within the cerebral cortex there are approximately 100,000 billion synapses. The behavior of the real nervous system is very complex and not yet fully known.

**Artificial Neuron - Perceptron**
Model of a Single Artificial Neuron (Perceptron)




Artificial Neuron Parameters

Following are the parameters related to an artificial Neuron.

Input Signals:
x1, x2, x3, x4, x5, x6,……, xp

Synaptic Weights:
wx1, wx2, wx3, wx4, wx5,……, wxp

Summing junction:
μ = ∑ xi * wxi

Bias:
b (depends on application, assigned by developer).

Activation Function
The activation function may be linear or non-linear.
A particular activation function is chosen to satisfy some specification of the problem that the neuron is attempting to solve.
A variety of activation functions have been proposed.

Activation/ Transfer / Unit Functions
Linear Functions
Simply output the weighted sum

Threshold Functions

Output low values

Until the weighted sum gets over a threshold

Then output high values

Equivalent of “firing” of neurons

Step function:

Output +1 if S > Threshold T

Output –1 otherwise

Hard limit activation function

This activation function sets the output of the neuron to 0 if the function argument is less than 0, or 1 if its argument is greater than or equal to 0. This function is generally used to create neurons that classify inputs into two distinct classes.

Hard-Limit Activation Function

$y = 0 \quad u < 0$

$y = 1 \quad u > 0$

Hard limit activation function - Example

Example: The input to a neuron is 3, and its weight is 1.8.

(a) What is the net input to the activation function?

(b) What is the output of the neuron if it has the hard limit transfer function?

Solution:

a) Net input = $u = 1.8 \times 3 = 5.4$

b) Neuron output = $f(u) = 1$.

Hard limit activation function - Example

Example: Given a two value input neuron with the following parameters: W = 1.2 and 3.4 and Input = -3 and 5, calculate the output of the perceptron for the hard limit transfer function.

Solution: net input to the activation function

$\mu = (1.2 * (-3)) + (3.4 * 5)$

$= 13.4$

Neuron output = $f(u) = 1.0$

Hard limit activation function - Example

Example: Given a two input neuron with the following parameters: W = 3 and 2, and x = -5 and 6. Calculate the neuron output for the symmetrical hard limit activation function.

Solution:

$\mu = (3(-5)) + (2x6)$

$= -3$

Output:

$y = f(u) = -1$

Example Perceptron

Categorization of 2x2 pixel black and white images

"bright" and "dark"

Representation of rule:

If the image contains 2, 3 or 4 white pixels, it should be considered "bright"

If it contains 0 or 1 white pixel, it is "dark"

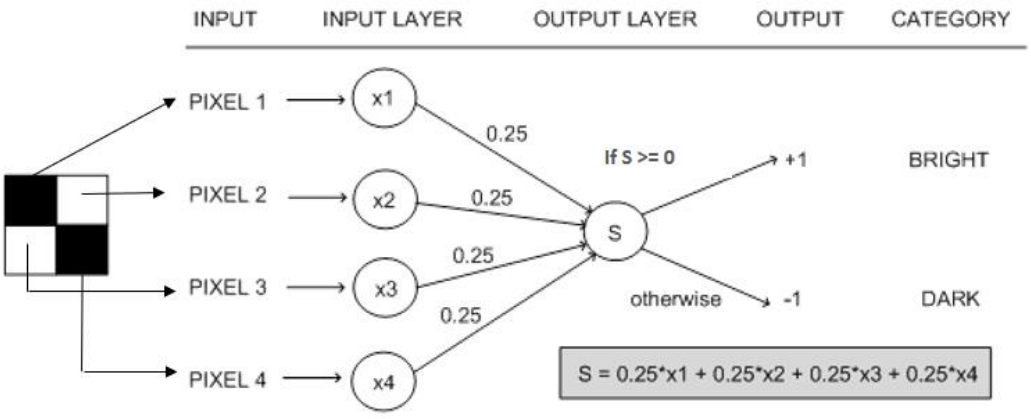
Perceptron architecture:

Four input units:

one for each pixel

One output unit:

+1 for white, -1 for dark



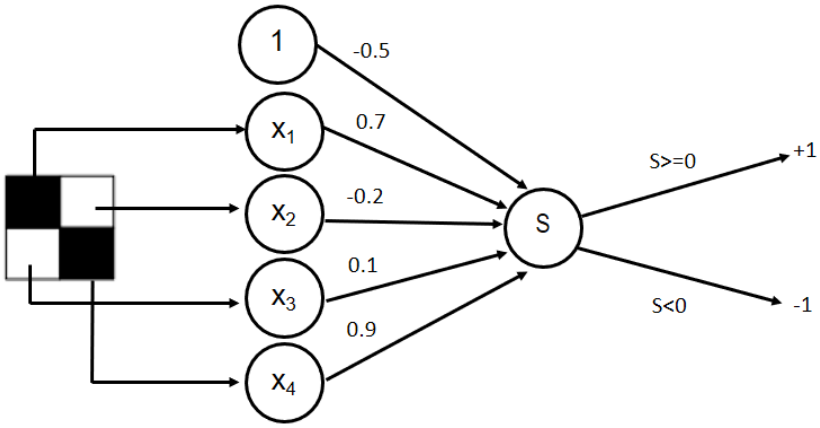

Example calculation: x1 = -1, x2 = 1, x3 = 1, x4 = -1

$S = 0.25 * (-1) + 0.25 * (1) + 0.25 * (1) + 0.25 * (-1) = 0$

The output from the perceptron is +1

The image is categorized as bright

**Example Perceptron - Learning Algorithm (Please refer to the solutions covered in the class)**




Categorize the provided 2x2 black and white image as bright or dark based on the same principle as the previous example.

Initially weights are set randomly
For each training example *E*
    Calculate the observed output from the perceptron: O(E)
    If the target output T(E) is different from O(E)
        Tweak all the weights so that O(E) gets closer to T(E)
        Tweaking is done by the perceptron training rule
This routine is done for every example *E*
    Repeat the cycle again (an 'epoch')
    Until the perceptron produces the correct output
        For all the examples in the training set (good enough)

When T(E) is different to O(E)
    Add on $\Delta i$ to weight wi
    Where $\Delta i = \eta (T(E) - O(E)) xi$
    Do this for every weight in the network

Interpretation:
    (T(E) - O(E)) can be -2 or +2
        -1 - (1) = -2 or (1 - (-1)) = 2
    So we can think of the addition of $\Delta i$ as the movement of the weight in a direction which will improve the networks performance with respect to E

    Multiplication by xi
        Moves it more if the input is bigger

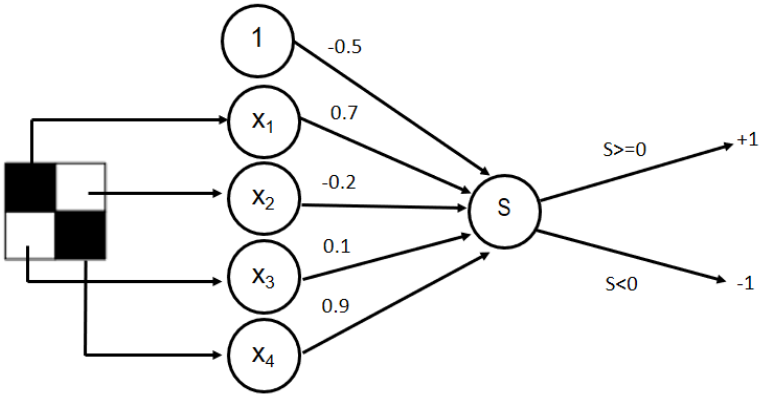
Learning Rate
η is called the learning rate
Usually set to something small (e.g., 0.1)

If a large movement is actually necessary for the weights to correctly categorise E
This will occur over time with multiple epochs

Solving the bright and dark image categorization using a learning rate of η = 0.1

Worked Example
Use this training example - E

Here, x1 = -1, x2 = 1
x3 = 1, x4 = -1




Let us propagate this information through the perceptron:
S = (-0.5 * 1) + (0.7 * -1) + (-0.2 * +1) + (0.1 * +1) + (0.9 * -1)
= -2.2

Hence the output O(E) = -1 ( S > 0 = +1 S <= 0 = -1)
But this should have been "bright" = +1
So T(E) = +1

Calculating the Error Values
$\Delta_0 = \eta(T(E)-O(E))x_0$
= 0.1 * (2) * (1) = 0.1 * (2) = 0.2

$\Delta_1 = \eta(T(E)-O(E))x_1$
= 0.1 * (2) * (-1) = 0.1 * (-2) = -0.2

$\Delta_2 = \eta(T(E)-O(E))\ x_2$

= 0.1 * (2) * (1)  = 0.1 * (2)  = 0.2

$\Delta_3 = \eta(T(E)-O(E))\, x_3$

= 0.1 * (2) * (1)  = 0.1 * (2)  = 0.2

$\Delta_4 = \eta(T(E)-O(E))\, x_4$

= 0.1 * (2) * (-1)  = 0.1 * (-2)  = -0.2

Make correction in the weights:

$w'_0 = -0.5 + \Delta_0$

= -0.5 + 0.2  = -0.3

$w'_1 = 0.7 + \Delta_1$

= 0.7 - 0.2  = 0.5

$w'_2 = -0.2 + \Delta_2$

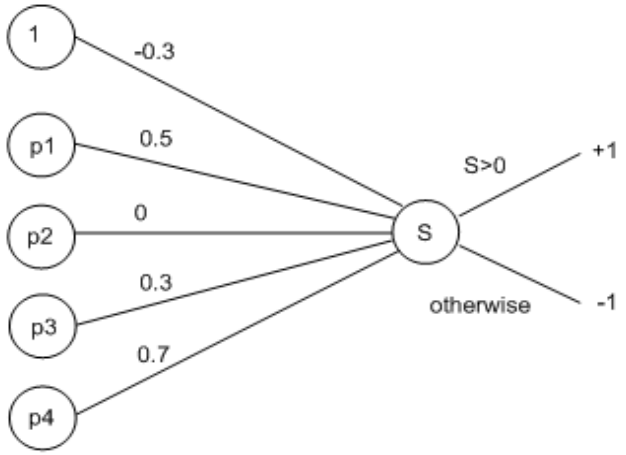
= -0.2 + 0.2  = 0

$w'_3 = 0.1 + \Delta_3$

= 0.1 + 0.2  = 0.3

$w'_4 = 0.9 + \Delta_4$

= 0.9 - 0.2  = 0.7




Calculate for the example, E, again:

S = (-0.3 * 1) + (0.5 * ((-1)) + (0 * 1) + (0.3 * 1) + (0.7 * (-1)) = -1.2

Still gets the wrong categorisation

But the value is closer to zero (from -2.2 to -1.2)

In a few epochs, this example will be correctly categorised. The correction and adjustments will continue till the network properly learns to recognize the image.

**Artificial Neural Network (ANN) architectures:**
Connecting several neurons in some specific manner yields an Artificial Neural Network (ANN).
The architecture of an ANN defines the network structure

- number of neurons in the network
- interconnectivity

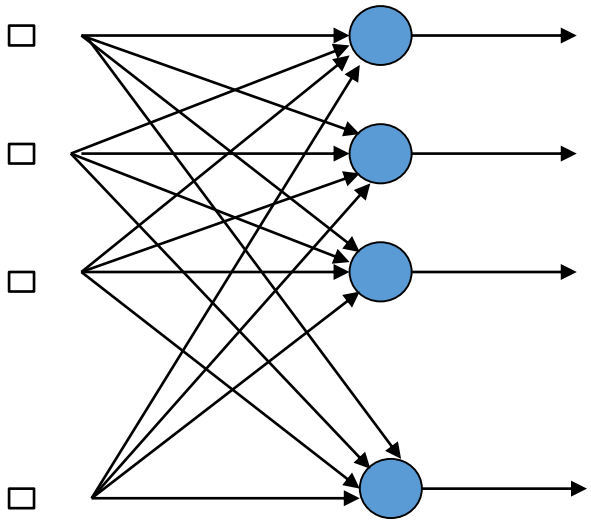
In a typical ANN architecture neurons are connected in layers and operate in parallel. The weights (strength of the connection) between the neurons are adapted during use to yield good performance. Each ANN architecture has its own learning rule.

Feedforward Neural Networks:
The term feedforward describes the direction in which information travels through the network.
Data always moves in one direction: Input → Processing → Output
When a signal passes through the network, it always propagates in the forward direction from one layer to the next layer, not to the other neurons in the same or the previous layer.

- Easy to implement
- The network has a layered architecture

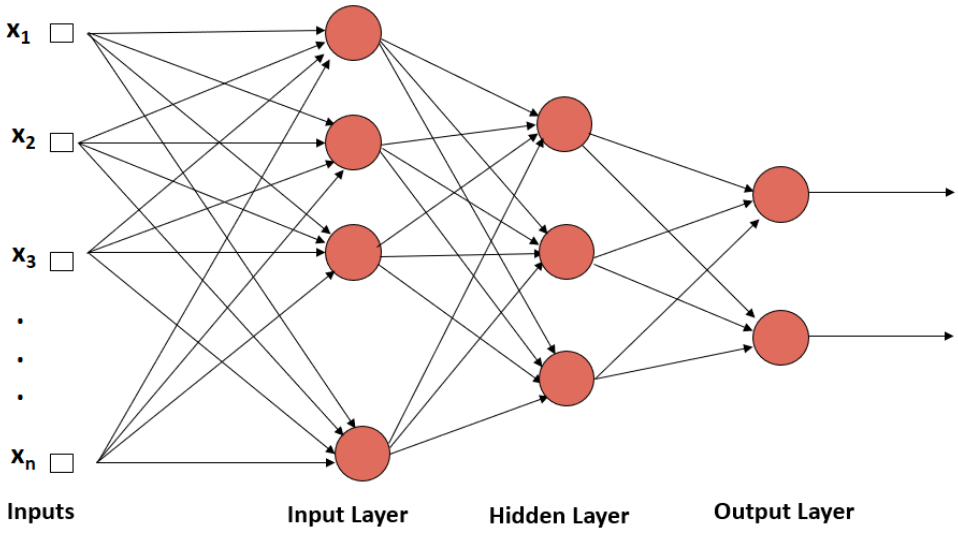
A layer consists of an arbitrary number of artificial neurons or nodes. In most cases, all the neurons in a particular layer contain the same activation function.
The neurons in different layers may have different activation functions.

**Single Layer Feedforward Neural Networks:**



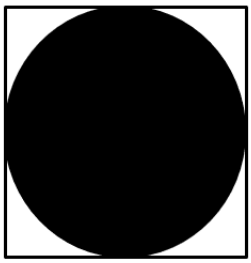
It consists of a single layer of output nodes.
The inputs are fed directly to the outputs via a series of weights.
The sum of the products of the weights and the inputs is calculated in each node: if the value is above some threshold (typically 0) the neuron fires and takes the activated value (typically 1) otherwise it takes the deactivated value (typically -1).

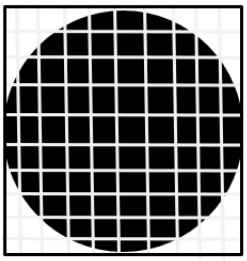
**Multi-Layer Feedforward Neural Networks:**

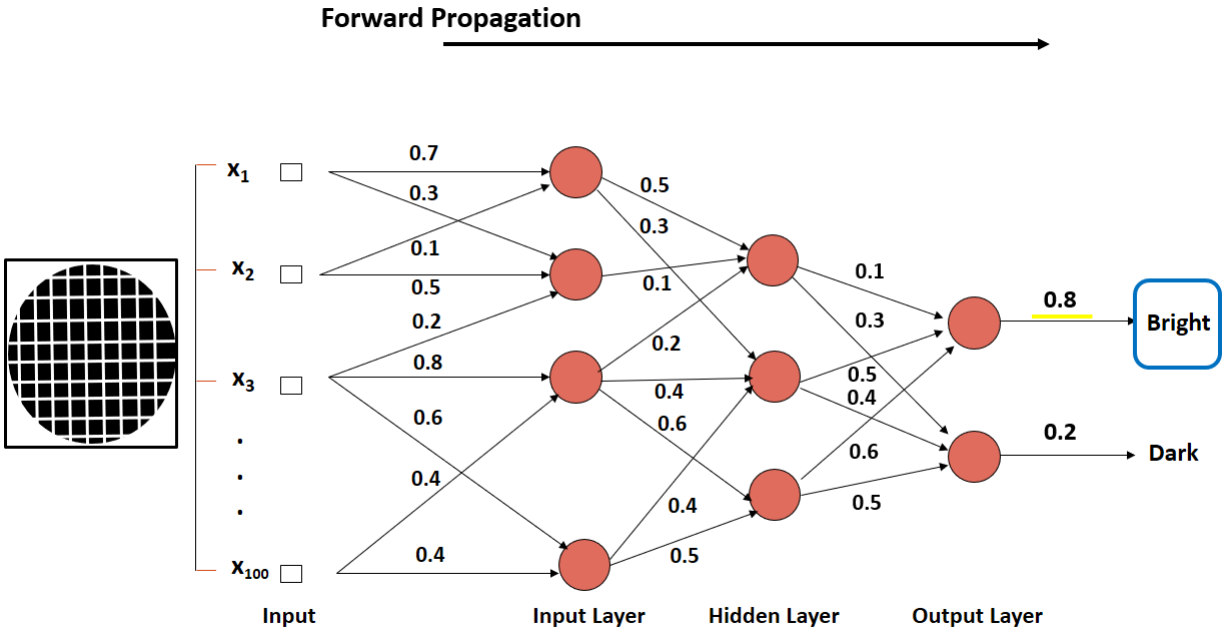


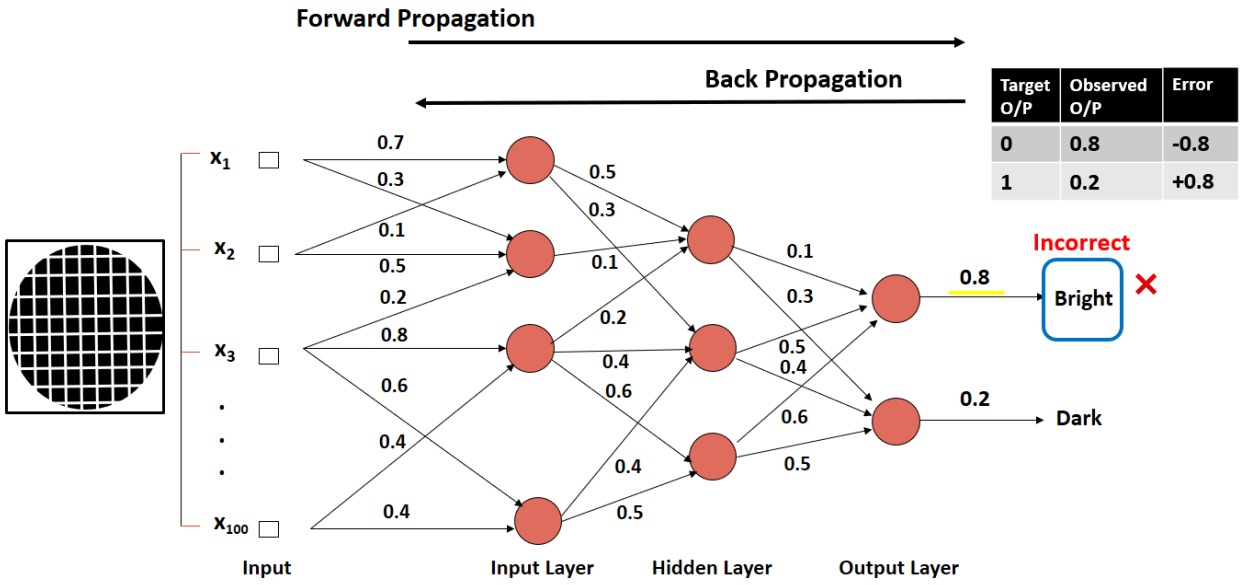

In a feedforward neural network, the first layer is called the input layer, and is the layer where the input patterns based on the data sample are fed in. Then follows one or more hidden layers, and as the name indicates these layers cannot be accessed from outside the network. The hidden layers enable the network to learn complex tasks by extracting progressively more meaningful features from the input patterns. The final layer is the output layer, which may contain one or more output neurons. This is the layer where the network decision, for a given input pattern, can be readout.

Working of a Neural Network
Categorisation of a 10 x 10 pixel black and white image as ‘bright’ or ‘dark’

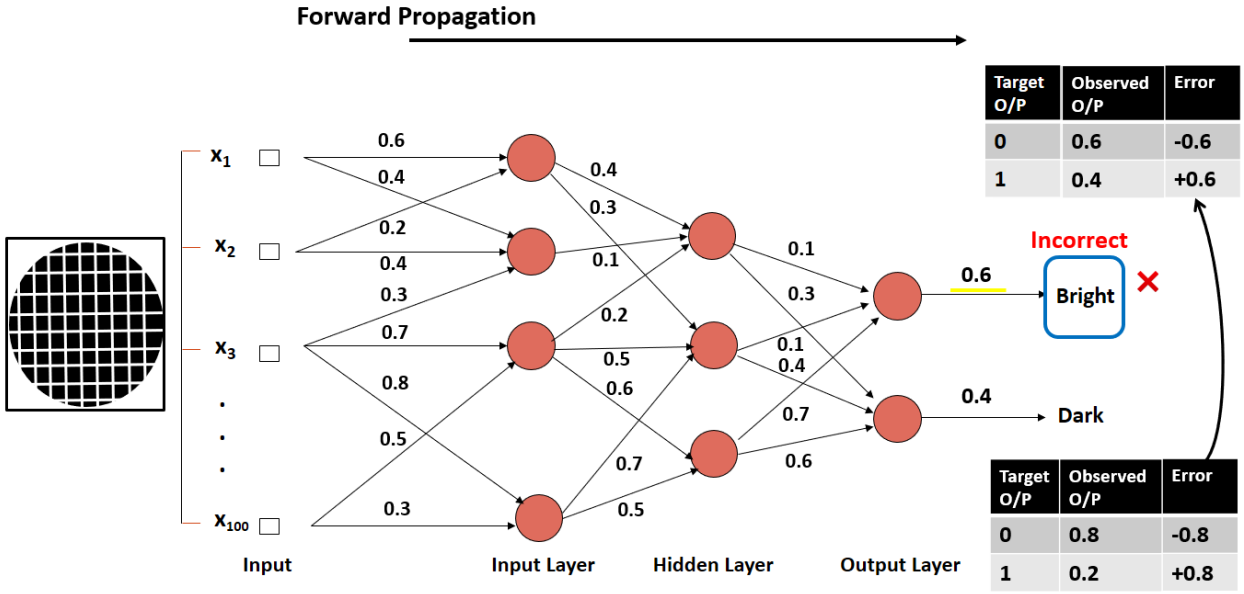







| Target O/P | Observed O/P | Error |
|---|---|---|
| 0 | 0.8 | -0.8 |
| 1 | 0.2 | +0.8 |


Forward Propagation
Target O/P
Observed O/P
Error
0
0.6
-0.6
1
0.4
+0.6
Incorrect
Bright
Dark
0.6
0.4
Target O/P
Observed O/P
Error
0
0.8
-0.8
1
0.2
+0.8
Input
Input Layer
Hidden Layer
Output Layer